

챕터 2. 게시판 CRUD

스프링이 내 코드를 어떻게 찾아 실행하는지 원리를 깨친 오픈이는 이제 본격적으로 스프링을 활용해 보기로 합니다. 첫 번째 목표는 웹 서비스의 가장 기본이 되는 **게시판** 만들기입니다. 새 글을 작성하고, 전체 목록을 조회하며, 작성한 글을 수정하거나 삭제하는 필수 기능들을 직접 구현해 볼 계획입니다.

게시판을 만드는 작업 중 중요한 것은 데이터베이스(DB)와의 통신입니다. 사용자가 작성한 글을 데이터베이스에 안전하게 저장해 두고, 누군가 목록이나 특정 글을 요청하면 보관해 둔 데이터를 꺼내 약속된 형식으로 돌려주는 것이 핵심입니다. 이제 오픈이와 함께 이 일련의 흐름을 코드로 완성해 보겠습니다.

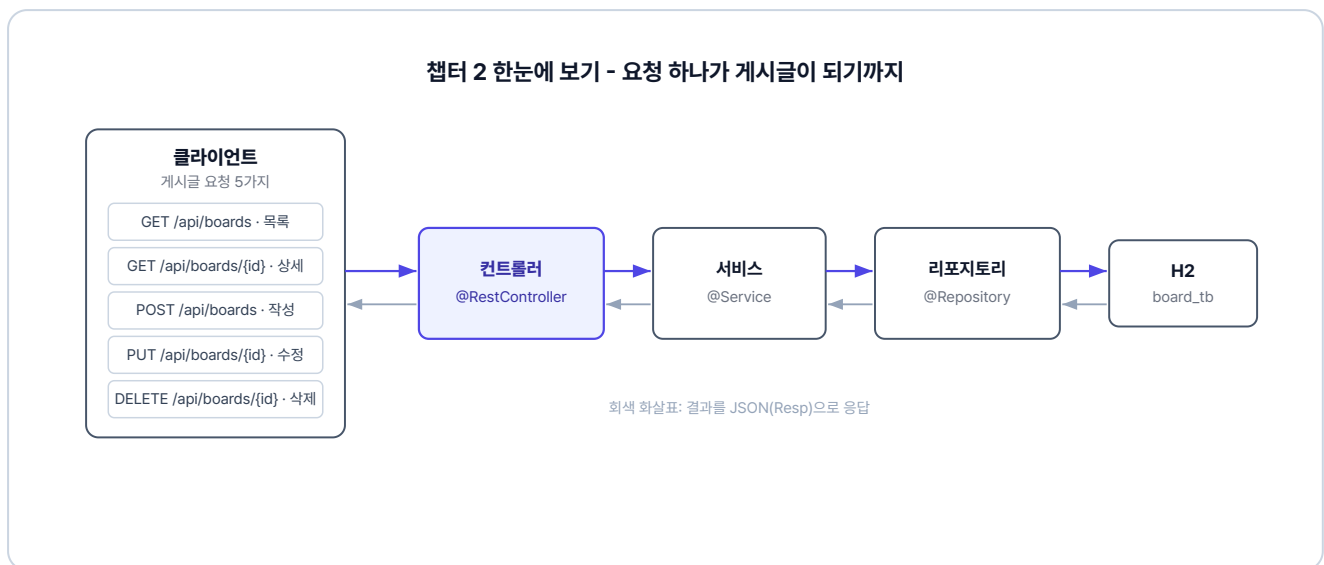


그림 2-1. 챕터 2의 실습 흐름

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다.

터미널 챕터 2 폴더로 이동

```
cd spring-start/ch02
```

ch02 폴더는 다음과 같이 구성되어 있습니다. 패키지 루트는 `com.metacoding.spring` 입니다.

ch02 파일 구조

```
spring-start/ch02/src/main/java/com/metacoding/spring/
├─ board/
│   ├── Board.java           # [작성] 게시물 클래스
│   ├── BoardController.java # [작성] REST 엔드포인트 5개
│   ├── BoardRepository.java # [작성] DB 저장·조회·삭제
│   └── BoardService.java     # [작성] 게시물 처리 흐름
└─ core/util/Resp.java      # [참고] 공통 응답 형식

spring-start/ch02/src/main/resources/
├─ application.properties   # [참고] H2·JPA 설정
└─ db/data.sql              # [참고] 더미 데이터

spring-start/ch02/src/test/java/com/metacoding/spring/board/
└─ BoardRepositoryTest.java  # [작성] 단위 테스트
```

핵심 로직 자리는 비어 있습니다. 챕터를 따라 코드를 채우고, 막히면 `spring-end` 의 완성 코드를 참고하세요.

이 파일들이 앞의 그림 2-1에서 본 세 층을 이룹니다. 각 클래스가 담당하는 역할은 다음과 같습니다.

클래스	역할
Board	게시글 한 건을 담는 클래스입니다. 데이터베이스의 게시물 표 한 줄에 대응합니다.
BoardRepository	게시글을 데이터베이스에 저장하고, 데이터베이스에서 조회하고 삭제합니다.
BoardService	목록, 상세, 작성, 수정, 삭제의 처리 흐름을 담당합니다.
BoardController	REST 요청을 받아 서비스로 넘기는 입구입니다.
Resp	모든 응답을 한 가지 형식으로 통일합니다.

2.1 REST API

REST API(Representational State Transfer API) 는 서버와 클라이언트가 데이터를 주고받는 방식입니다. 클라이언트는 주소(URI)로 무엇을 원하는지 가리키고, HTTP 메서드로 그것에 무엇을 할지 나타냅니다.

이때 주소가 가리키는 것을 **자원(Resource)** 이라고 합니다. 예를 들어 게시판 서버라면 게시글 목록과 1번 게시글이 각각 자원이고, `/boards` 와 `/boards/1` 이 그 주소가 됩니다.

2.1.1 REST API 탄생 배경

REST API가 왜 지금의 모습이 되었는지, 웹의 발전 과정을 세 단계로 거슬러 올라가 보겠습니다.

초창기의 웹 서버는 미리 만들어 둔 문서나 이미지 같은 정적 자원을 그대로 돌려주는 역할만 했습니다. 브라우저가 특정 주소를 요청하면, 서버는 그 주소에 있는 파일을 찾아 있는 그대로 보내주었습니다.

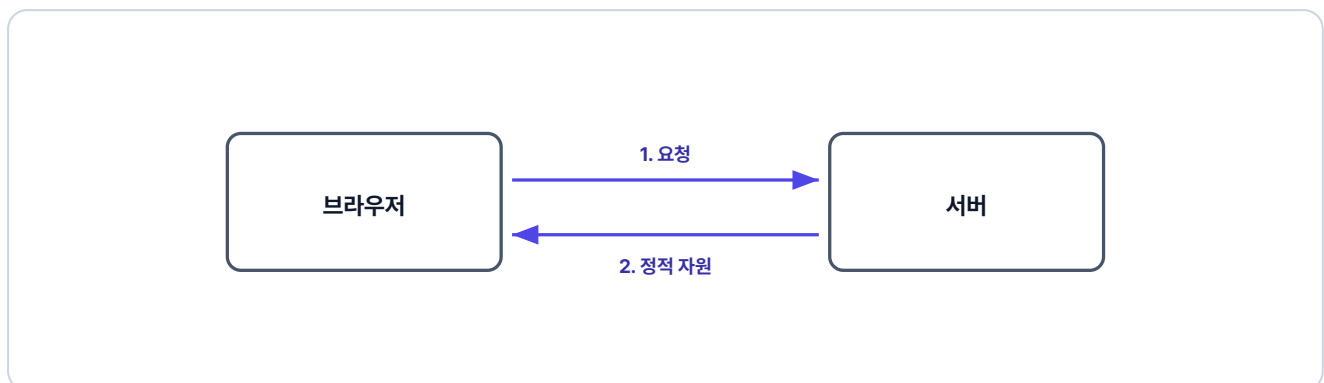


그림 2-2. 초창기 서버의 정적 응답

이후 인터넷이 발전하면서 서버는 단순한 파일 전달을 넘어 WAS(웹 애플리케이션 서버) 형태로 진화했습니다. 요청이 들어올 때마다 서버가 상황에 맞는 HTML 화면을 새로 만들어 응답하기 시작한 것입니다. 하지만 이때까지만 해도 서버가 만든 HTML 화면을 받아보는 대상은 오직 **웹 브라우저**뿐이었습니다.

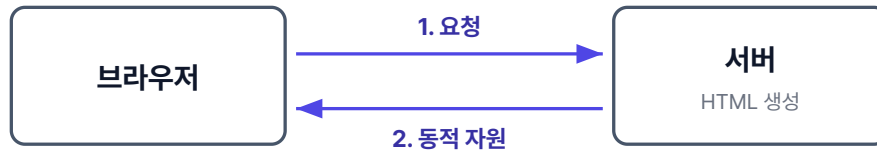


그림 2-3. WAS의 동적 화면 생성

문제는 스마트폰 앱, 스마트 TV, 심지어 다른 서버들까지 등장하면서 시작됐습니다. 다양한 기기들이 서버에 동일한 데이터를 요청하기 시작했는데, 브라우저 전용인 **HTML 화면**을 통째로 넘겨주는 것은 이들에게 전혀 맞지 않았습니다.

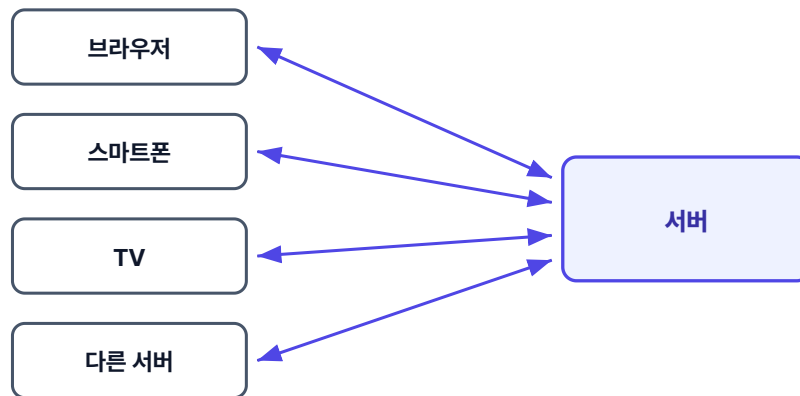


그림 2-4. REST API로 자원에 접근하는 여러 기기

결국 서버는 무거운 화면(HTML) 대신 **데이터** 그 자체만 전달하는 방향으로 변화했습니다. 서버가 HTML이 아닌 JSON이나 XML 같은 데이터를 반환하고, 클라이언트가 이를 받아 화면 구성이나 로직을 처리하는 방식이 REST API입니다.

REST API 방식을 활용하면 브라우저뿐 아니라 스마트폰, TV, IoT 기기, 다른 서버에서도 자원에 접근할 수 있어, 다양한 플랫폼에서 활용 가능한 서버로 확장할 수 있습니다.

2.1.2 JSON

그렇다면 서버는 어떤 형식으로 데이터를 넘겨줄까요? 이때 가장 널리 쓰이는 표준 형식이 바로 JSON입니다.

JSON(JavaScript Object Notation) 은 데이터를 **키(Key)** 와 **값(Value)** 의 쌍으로 묶어 표현하는 텍스트 형식입니다. 구조가 단순해서 기계뿐만 아니라 사람도 직관적으로 읽고 이해하기 쉽다는 장점이 있습니다. 게시글 한 건을 JSON으로 표현하면 이런 모습입니다.

```
{
  "id": 1,
  "title": "첫 번째 게시글",
  "content": "안녕하세요"
}
```

2.1.3 메서드와 주소

REST API의 핵심은 **무엇을(주소)**, **어떻게(메서드)** 할 것인가를 명확하게 나누는 데 있습니다. 주소(URI)가 조작할 대상(자원)을 가리킨다면, 그 대상에 어떤 작업을 할지는 HTTP 메서드가 결정합니다.

게시판을 예로 들면, 게시글이라는 자원을 다룰 때 주로 다음 네 가지 메서드를 사용합니다.

메서드	하는 일	예
GET	조회한다	게시글 목록을 가져온다
POST	새로 만든다	게시글을 작성한다
PUT	수정한다	게시글 내용을 고친다
DELETE	삭제한다	게시글을 지운다

이처럼 HTTP 메서드를 통해 **행위(어떻게)** 를 명확히 분리하고 나면, 주소에는 온전히 **자원(무엇을)** 만 남게 됩니다.

REST API에서는 다른 개발자가 주소만 보고도 어떤 자원을 다루는지 직관적으로 예측할 수 있도록, 자원의 이름을 지을 때 지켜야 할 몇 가지 권장 규칙을 두고 있습니다.

규칙	권장	피할 것
주소의 영어는 소문자로 쓴다	/boards	/Boards
주소에 행위를 넣지 않고 메서드로 표현한다	PUT /boards/1	/boards/1/put

규칙	권장	피할 것
자원은 단수형이 아닌 복수형으로 쓴다	<code>/boards/1</code>	<code>/board/1</code>
긴 단어는 하이픈으로 구분한다	<code>/check-username</code>	<code>/check_username</code>
확장자를 붙이지 않고 헤더에 타입을 담는다	<code>/users + 헤더 application/json</code>	<code>/users.json</code>

2.1.4 요청과 응답

클라이언트와 서버가 통신할 때는 미리 정해진 규격에 맞춰 메시지를 주고받습니다. 이 메시지는 클라이언트가 보내는 **요청(Request)** 과 서버가 돌려주는 **응답(Response)** 으로 나뉘며, 둘 다 세 부분으로 이루어집니다.

먼저 HTTP 요청의 세 부분은 **요청 라인(Request Line)**, **헤더(Header)**, **바디(Body)** 입니다.

- **요청 라인:** HTTP 메서드와 주소가 들어갑니다.
- **헤더:** 바디에 담긴 데이터의 형식(`Content-Type`) 등 서버가 참고할 부가 정보가 담깁니다.
- **바디:** 서버로 보낼 실제 데이터가 담깁니다.

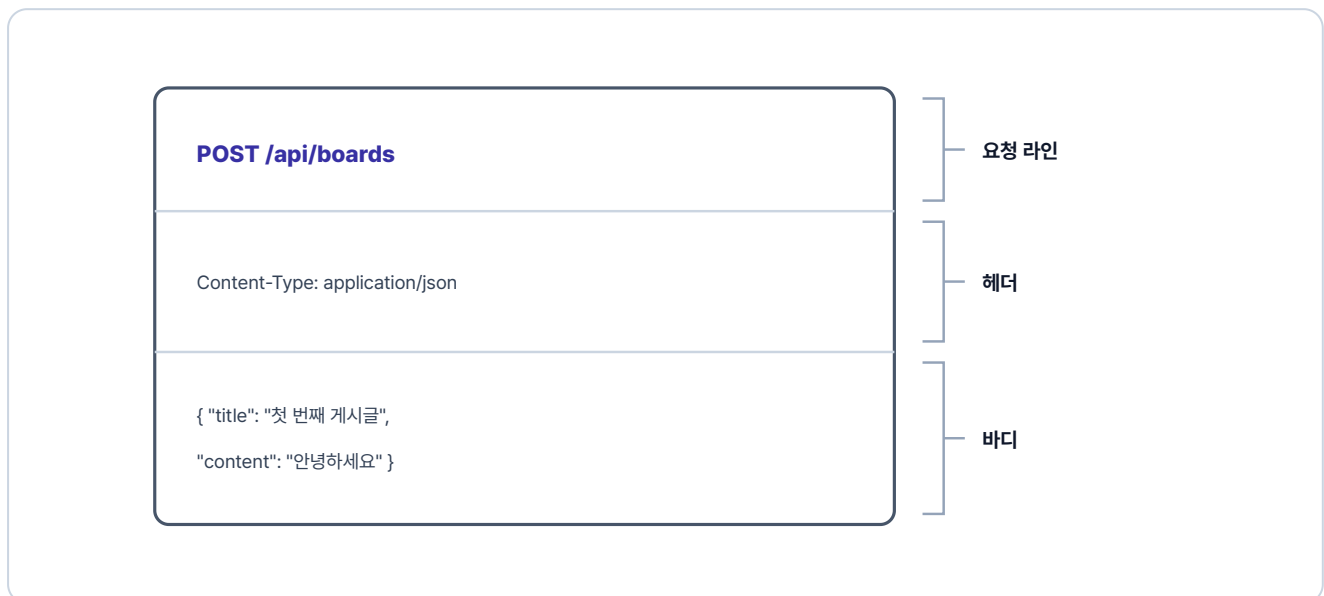


그림 2-5. HTTP 요청의 구조

요청을 받아 처리한 서버는 마찬가지로 세 부분으로 나뉜 HTTP 응답을 돌려줍니다. 다만 첫 줄은 요청 라인이 아니라 **응답 라인(Status Line)** 입니다.

- **응답 라인:** 요청이 성공했는지(200 OK), 아니면 문제가 생겼는지를 나타내는 **상태 코드(Status Code)** 가 들어갑니다.
- **헤더:** 클라이언트에게 돌려주는 데이터의 형식 등 부가 정보가 담깁니다.
- **바디:** 클라이언트가 요청한 실제 데이터가 담깁니다. 게시글 하나를 요청했다면 앞에서 본 JSON이 이 바디에 담겨 있습니다.

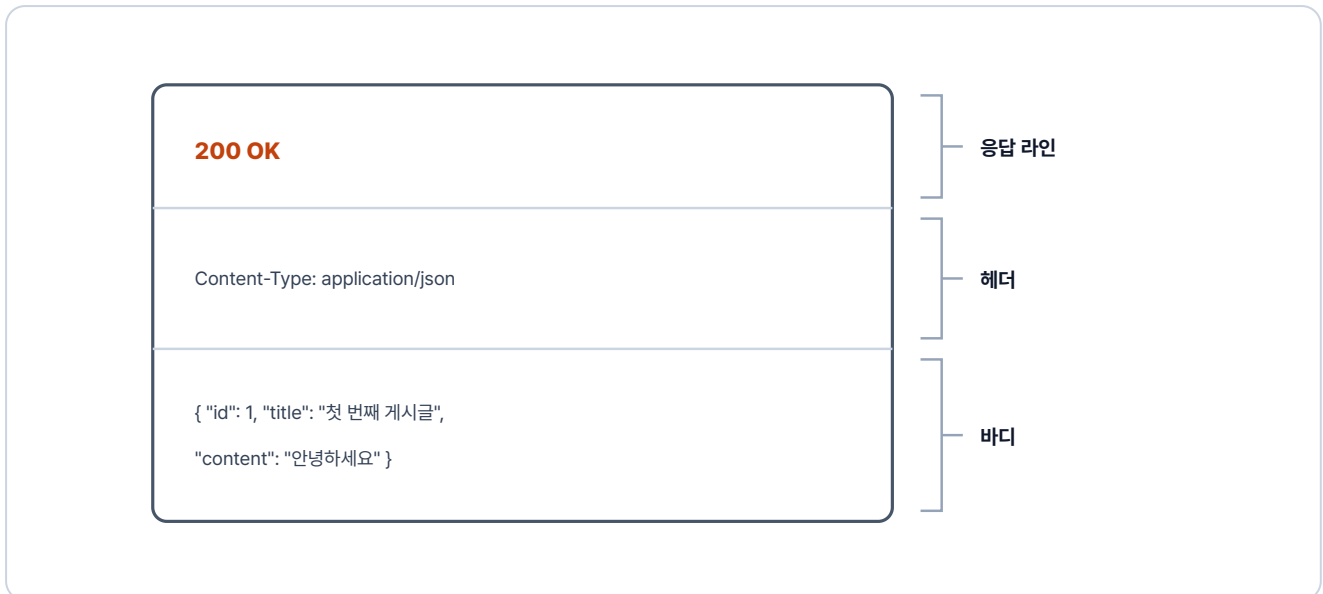


그림 2-6. HTTP 응답의 구조

2.2 프로젝트 생성과 구조

스프링 부트는 스프링을 사용하는 데 필요한 것들을 미리 묶어 둔 도구로, 내장 서버를 갖추고 설정을 맞추는 일을 대신 해 줍니다. 개발자는 명령 한 줄로 서버를 실행하고 기능부터 만들면 됩니다.

2.2.1 프로젝트 생성

이 책의 실습 방식

본격적인 실습은 미리 준비해 둔 **spring-start** 저장소에서 진행합니다. 다만 첫 프로젝트 생성 과정은 직접 눈으로 확인해 보는 것이 좋으므로 여기서는 프로젝트를 만드는 절차만 먼저 살펴보겠습니다.

먼저 자바와 스프링을 다룰 확장 프로그램을 설치합니다. 상단 탭에서 View > Extensions를 선택합니다.

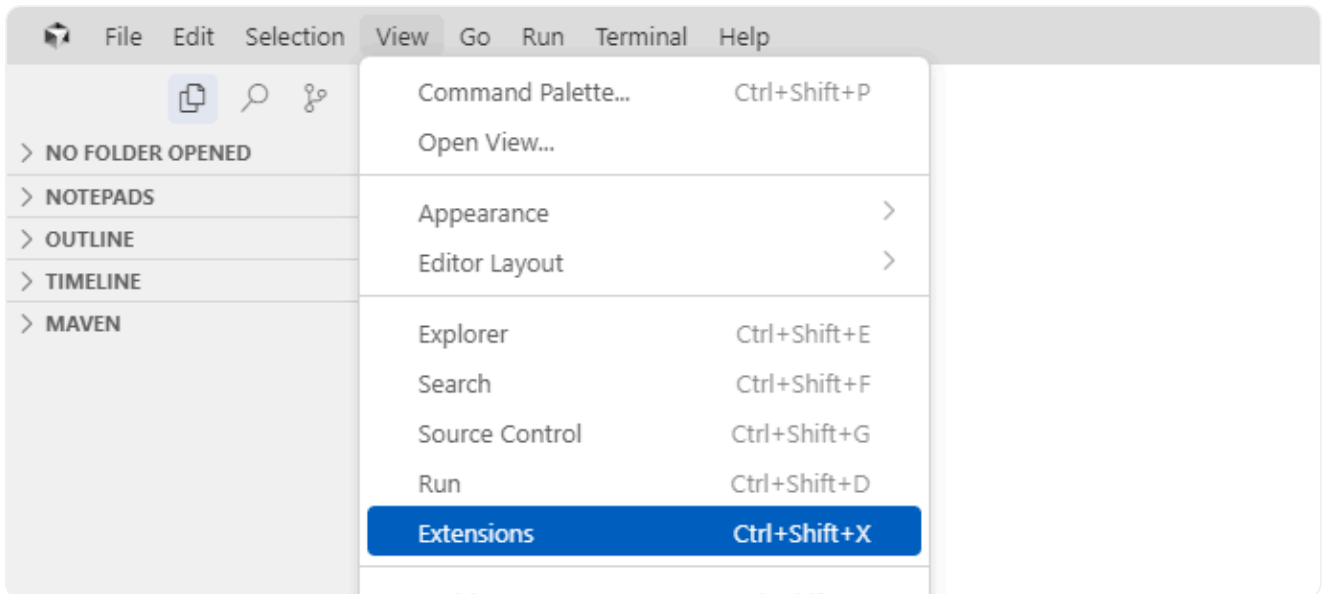


그림 2-7. Extensions 메뉴 선택

검색창에 java를 넣어 Extension Pack for Java를 설치합니다.

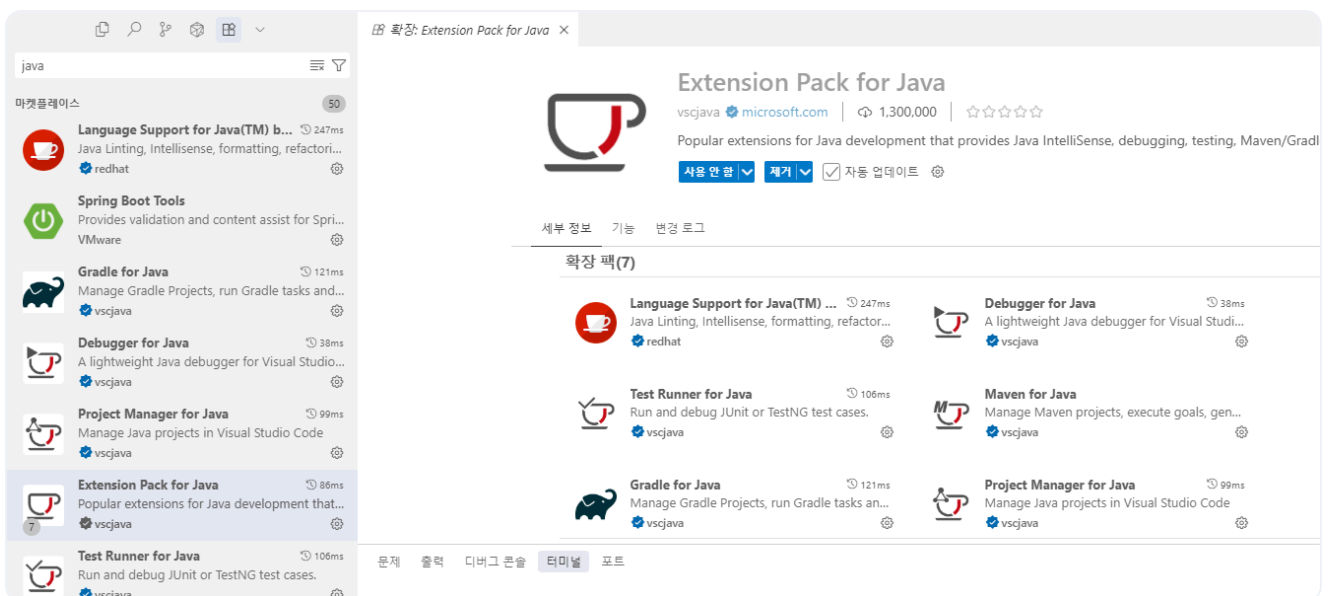


그림 2-8. Extension Pack for Java

이어서 spring으로 검색해 Spring Boot Extension Pack을 설치합니다. 이 확장에 스프링 프로젝트를 만들어 주는 Spring Initializr가 들어 있습니다.

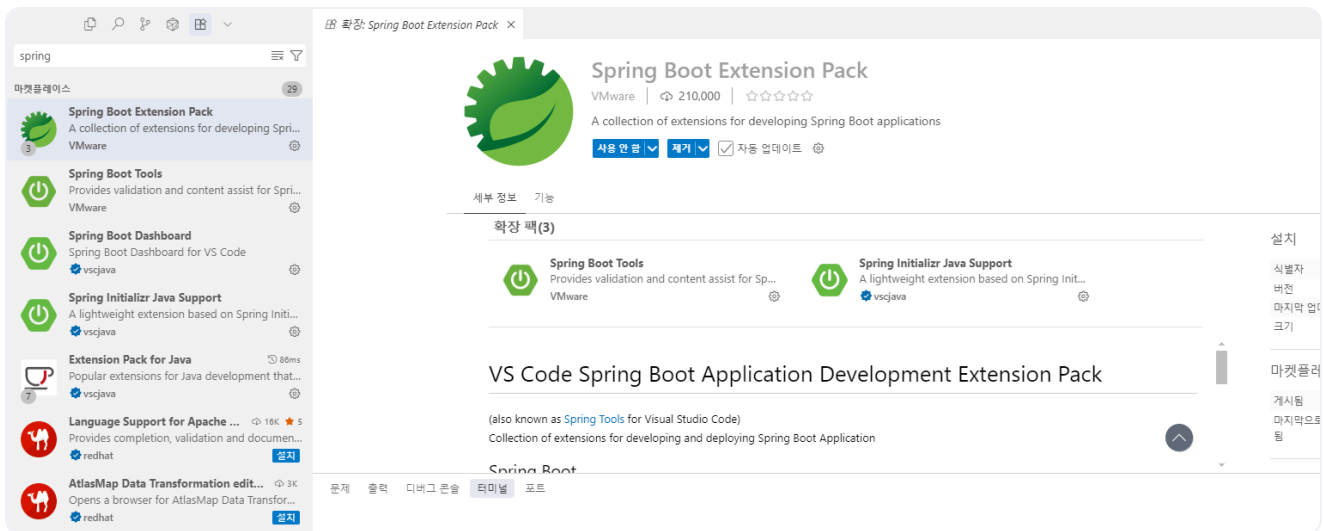


그림 2-9. Spring Boot Extension Pack

설치가 끝나면 View > Command Palette를 엽니다.

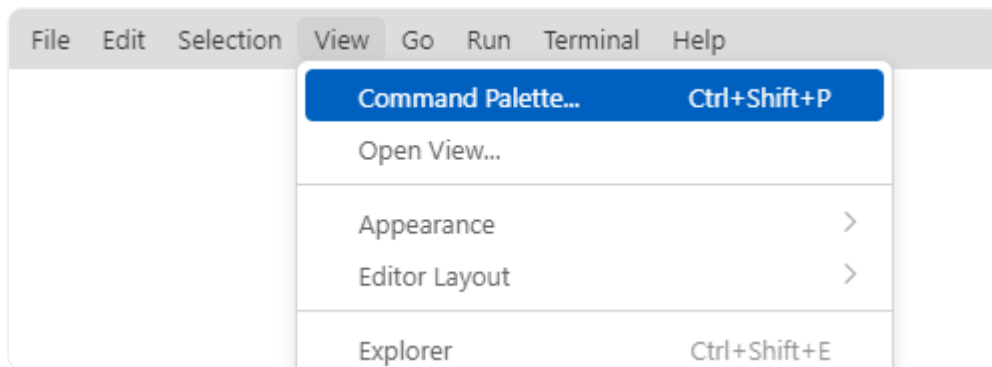


그림 2-10. Command Palette 선택

spring을 입력해 Spring Initializr: Create a Gradle Project를 실행합니다.

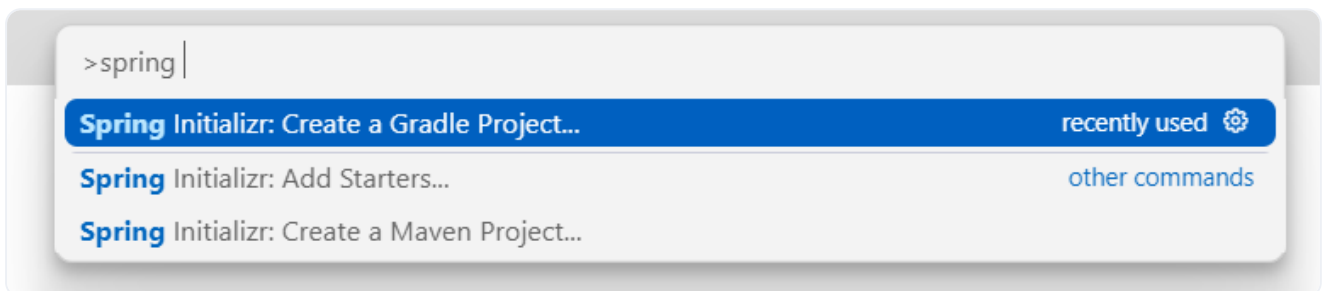


그림 2-11. Gradle 프로젝트 생성 명령

이후 묻는 항목에 이 값들을 넣습니다.

항목	값
스프링 부트 버전	4.0.7
언어	Java
Group Id	com.metacoding
Artifact Id	spring-ch02
Packaging	JAR
자바 버전	21

마지막으로 프로젝트에 넣을 의존성을 고릅니다.

의존성	역할
Spring Web	REST 요청을 받는 웹 계층과 내장 웹 서버가 들어 있습니다
Spring Data JPA	객체와 테이블을 잇는 JPA와 하이버네이트가 들어 있습니다
H2 Database	메모리에서만 동작하는 실습용 데이터베이스입니다
Lombok	게터·Setter 같은 반복 코드를 대신 만들어 줍니다

생성이 끝나면 이런 구조가 만들어집니다.

`build.gradle`에 방금 고른 의존성이 적혀 있고, 코드는 `src` 아래에 들어갑니다.

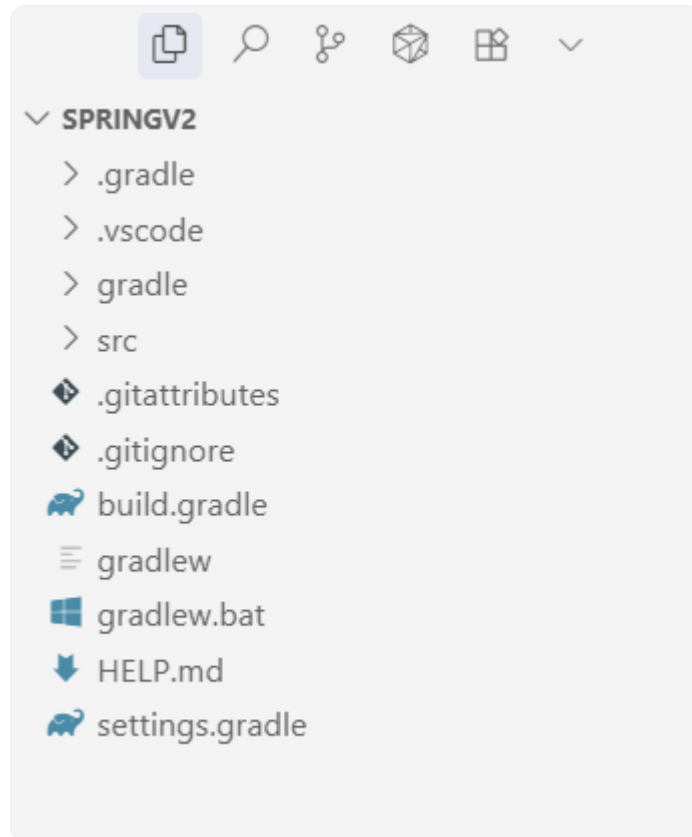


그림 2-12. 스프링 부트 프로젝트 구조

2.3 하이버네이트

220V 플러그는 110V 콘센트에 그대로 꽂히지 않습니다. 규격이 다르기 때문에 사이에 어댑터가 필요합니다. 자바와 데이터베이스도 데이터를 담는 구조가 서로 달라서, 객체를 표에 그대로 넣을 수 없습니다.

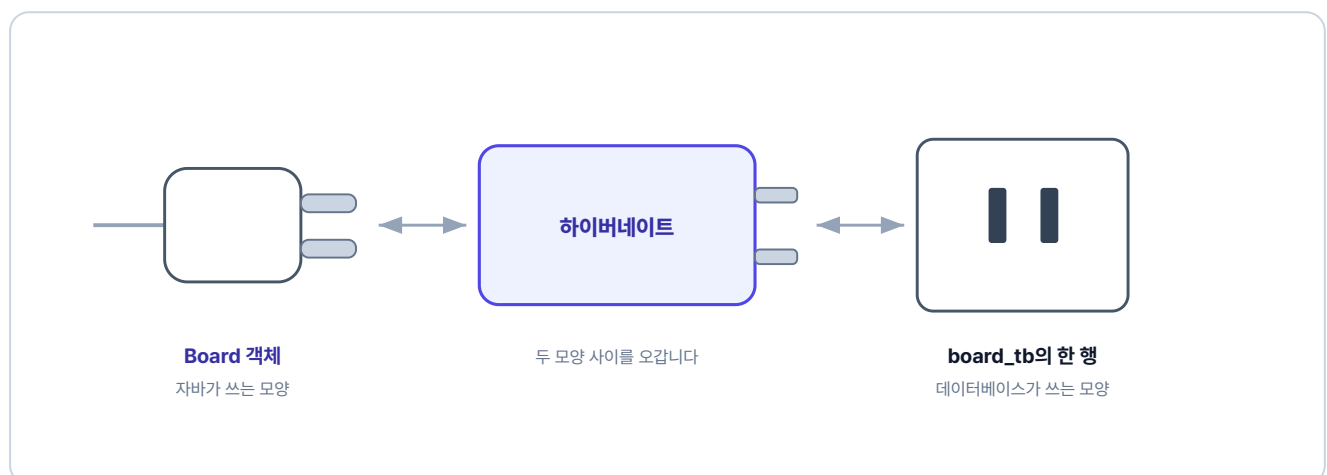


그림 2-13. 어댑터를 끼운 플러그와 콘센트

앞으로 만들 게시글 하나는 자바에서 **Board** 객체이고, 데이터베이스에서는 **board_tb** 테이블의 한 행입니다. 두 구조의 차이는 이렇습니다.

구분	데이터베이스 (RDB)	자바 (Java)
구조	행과 열로 이루어진 2차원의 평면적인 표입니다. 각 칸에는 숫자나 문자 같은 단순한 값만 들어갑니다.	상태(필드)와 행위(메서드)를 함께 가지는 입체적인 구조입니다.
연관관계	데이터 간의 관계를 외래 키 (Foreign Key) 를 통해 맺습니다. 한 칸에 표나 배열을 통째로 넣을 수 없습니다.	객체 안에 다른 객체를 품거나, List나 Map 같은 컬렉션을 담아 훨씬 복잡한 관계를 표현할 수 있습니다.
상속	부모와 자식이라는 상속의 개념 자체가 없습니다.	부모 클래스의 특징을 물려받는 상속 과 다형성 이 핵심입니다.

이러한 구조적 차이 때문에, 객체를 저장하거나 꺼낼 때는 매번 값을 일일이 옮겨 담는 변환 작업이 필요합니다. 마치 어댑터처럼 이 간극을 메워주는 기술을 **ORM(Object-Relational Mapping)** 이라고 합니다. 자바에서는 이 ORM이 갖춰야 할 기능을 **JPA(Java Persistence API)** 라는 규칙으로 정해두었고, 이 규칙대로 만든 대표적인 엔진이 바로 **하이버네이트(Hibernate)** 입니다. 우리가 앞서 추가한 Spring Data JPA 의존성에는 이 하이버네이트가 기본적으로 포함되어 있습니다.

2.4 데이터베이스와 엔티티

2.4.1 데이터베이스 설정

프로젝트의 환경 설정은 `resources/application.properties` 파일에서 관리합니다. 서버 포트나 데이터베이스 연결 정보, JPA 동작 방식 등을 이곳에 지정합니다.

참고 `resources/application.properties`. 프로젝트 환경 설정

```
# 8080 포트로 실행하고, 요청과 응답의 문자를 UTF-8로 처리해 한글이 깨지지 않게 한다
server.port=8080
spring.servlet.encoding.charset=UTF-8
spring.servlet.encoding.enabled=true
spring.servlet.encoding.force=true
```

```

# 메모리에서 동작하는 H2에 sa 계정으로 접속한다
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:mem:test
spring.datasource.username=sa
spring.datasource.password=

# 브라우저에서 H2 콘솔로 테이블을 확인할 수 있게 한다
spring.h2.console.enabled=true

# 시작 시점에 실행할 SQL 파일의 위치
spring.sql.init.data-locations=classpath:db/data.sql

# 테이블이 만들어진 뒤에 위 파일을 실행한다
spring.jpa.defer-datasource-initialization=true

# 스프링 실행 시 @Entity가 붙은 클래스를 테이블로 자동 생성한다
spring.jpa.hibernate.ddl-auto=create

# JPA가 만든 SQL을 콘솔에 보기 좋게 출력한다
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

실습에 사용할 데이터베이스는 **H2**입니다. 자바로 만들어진 가벼운 관계형 데이터베이스로, 별도의 설치 과정이 필요 없습니다. 다만 메모리 기반으로 동작하기 때문에 서버를 종료하면 저장된 데이터는 모두 사라집니다.

ddl-auto=create는 개발 단계에서만

ddl-auto=create는 스프링을 실행할 때마다 테이블을 새로 생성하는 설정입니다. 운영 중인 데이터베이스에 두면 재시작 한 번으로 데이터가 사라지므로, 개발 단계에서만 사용합니다.

2.4.2 엔티티

이어서 게시글을 표현하는 **엔티티(Entity)**를 만들어 보겠습니다. 엔티티란 데이터베이스의 테이블과 매핑되는 자바 클래스를 말합니다. 즉, 엔티티 클래스로 생성한 객체 하나가 데이터베이스 테이블의 한 행(Row)이 됩니다.

`board/Board.java`를 열어 아래와 같이 작성합니다.

실습 1 board/Board.java. 게시글 엔티티

```

@Data // 롬복(Lombok). 게터·세터·toString을 컴파일 시점에 대신 만든다
@Entity

```

```
@Table(name = "board_tb") // 이 클래스를 board_tb 테이블에 매핑한다
public class Board {
    @Id // 기본 키. DB가 자동으로 1씩 증가시켜 채운다
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String title;
    private String content;

    @CreationTimestamp // 저장 시점의 현재 시간을 자동으로 기록한다
    private LocalDateTime createdAt;
}
```

애플리케이션을 실행하면 하이버네이트가 이 클래스를 읽어 **board_tb** 테이블을 생성합니다. 앞으로 게시글을 저장하고 조회할 때 필요한 SQL도 하이버네이트가 생성합니다.

2.4.3 더미 데이터

실습을 위해 미리 넣어 두는 가상의 데이터를 **더미 데이터(Dummy Data)** 라고 합니다.

참고 [resources/db/data.sql](#). 게시글 더미 데이터

```
insert into board_tb (title, content, created_at) values ('title1', 'content1', now());
insert into board_tb (title, content, created_at) values ('title2', 'content2', now());
```

터미널에 `./gradlew bootRun` 을 입력해 스프링을 실행합니다.

[illegible]

그림 2-14. 스프링 실행 결과

브라우저에서 `http://localhost:8080/h2-console` 을 입력하면 H2 콘솔에 접속할 수 있습니다.
접속할 User Name과 Password는 `application.properties` 에서 설정한 값입니다.

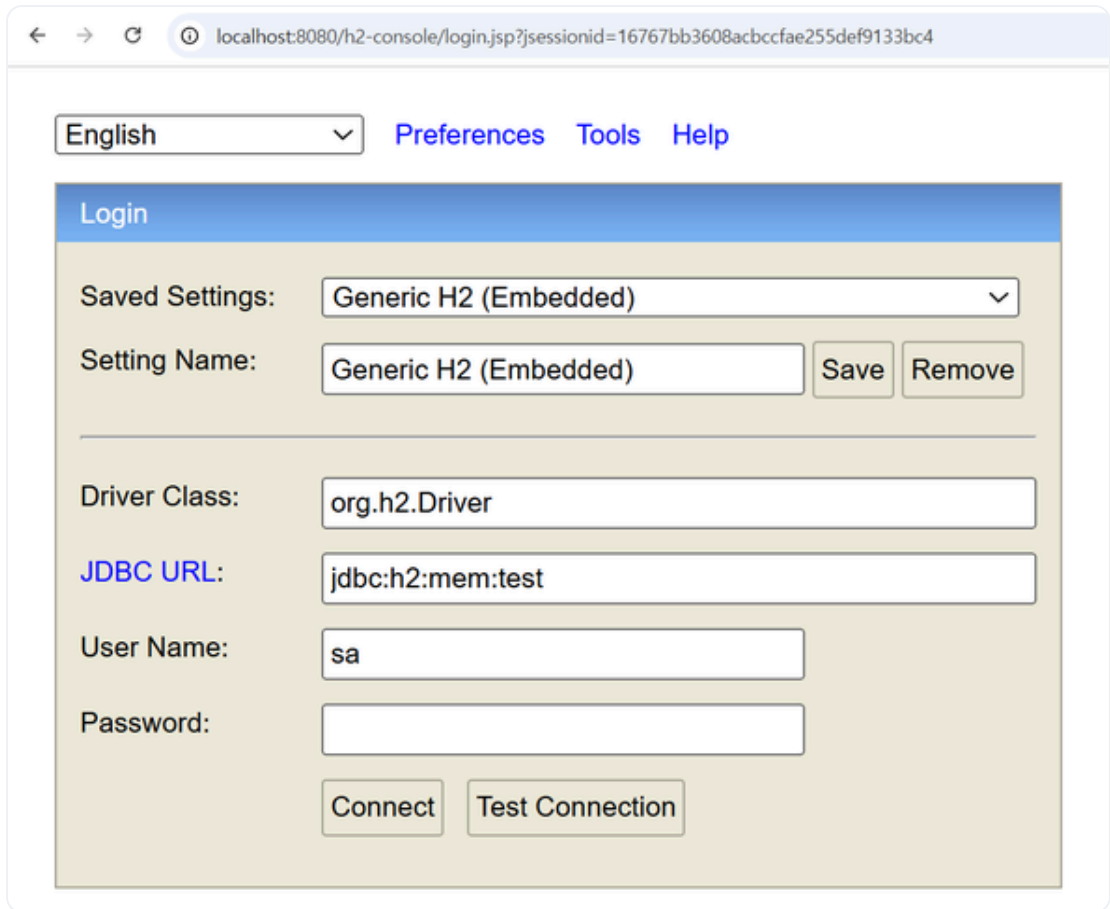


그림 2-15. H2 콘솔 로그인 화면

H2 콘솔에서 **board_tb** 테이블을 확인할 수 있습니다.

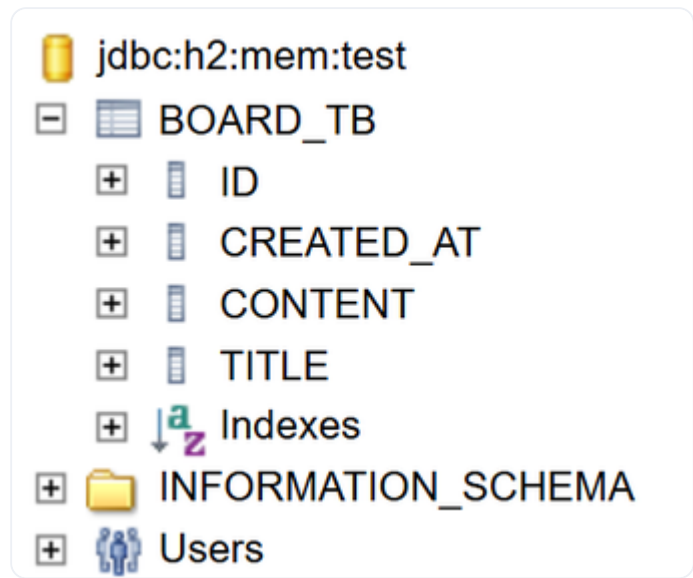


그림 2-16. H2 콘솔의 board_tb 테이블

H2 콘솔에서는 SQL문을 실행할 수 있습니다. SELECT 쿼리로 테이블을 조회하면 `data.sql`에 넣어둔 데이터가 들어와 있는 것을 확인할 수 있습니다.

RunRun SelectedAuto completeClearSQL statement:

SELECT * FROM BOARD_TB

SELECT * FROM BOARD_TB;

ID	CREATED_AT	CONTENT	TITLE
1	2025-09-11 21:51:56.910579	content1	title1
2	2025-09-11 21:51:56.91258	content2	title2

(2 rows, 3 ms)

그림 2-17. board_tb 조회 결과

자바의 카멜 케이스와 DB의 스네이크 케이스

자바는 단어의 첫 글자를 대문자로 적는 **카멜 표기법(createdAt)** 을 주로 사용하고, 데이터베이스는 단어 사이를 밑줄로 잇는 **스네이크 표기법(created_at)** 을 관례로 사용합니다. 하이버네이트는 엔티티 필드의 카멜 표기를 DB의 스네이크 표기 컬럼명으로 자동 변환해 주므로, 개발자는 자바의 네이밍 규칙만 신경 쓰면 됩니다.

2.5 3계층 아키텍처

게시판에 필요한 모든 코드를 하나의 클래스에 작성해도 프로그램은 정상적으로 돌아갑니다. 하지만 요청과 응답을 다루는 코드, 비즈니스 로직을 처리하는 코드, 데이터베이스에 접근하는 코드가 한 곳에 있으면 어디서 에러가 났는지 찾기도 힘든 복잡한 코드가 됩니다.



그림 2-18. 한 클래스에 섞인 코드

이런 문제를 막기 위해, 성격이 같은 코드끼리 층을 나누어 역할을 분리합니다. 이렇게 하면 각 클래스가 맡은 책임이 하나로 뚜렷해져 시스템에 변화가 생겼을 때 다른 코드에 미치는 영향을 최소화할 수 있습니다.



그림 2-19. 계층으로 나눈 코드

스프링 웹 개발에서는 이렇게 역할에 따라 코드를 3개의 층으로 나누는 **3계층 아키텍처(3-Layer Architecture)** 를 주로 사용합니다.

층	하는 일
컨트롤러(Controller)	HTTP 요청 주소를 받아 서비스를 호출하고, 돌려받은 결과를 응답합니다
서비스(Service)	리포지토리를 호출해 요청을 완성하는 비즈니스 로직 을 수행하고, 그 작업을 하나의 트랜잭션으로 처리합니다
리포지토리(Repository)	엔티티를 데이터베이스에서 조회하고 저장·수정·삭제합니다

Spring MVC와 JSON 응답

스프링에서 웹 요청을 다루는 기술을 **Spring MVC**라고 부릅니다. 본래는 화면(View), 요청 처리(Controller), 데이터(Model)를 나누는 전통적인 웹 개발 방식에서 유래한 이름입니다. 하지만 이 책에서는 별도의 화면을 만들지 않고 순수하게 데이터만 제공하는 API 서버를 만듭니다. 따라서 View 계층 없이, 컨트롤러가 반환한 자바 객체가 JSON 문자로 변환되어 응답 본문(Body)에 실립니다.

2.6 공통 응답

core 폴더는 프로젝트에 필요한 설정 파일, 도구 파일 등을 모아 놓은 폴더입니다.

Resp 클래스는 API 응답 구조를 정하는 역할을 합니다. 요청이 성공하면 **status** 를 200으로 두고 응답 데이터를 **body** 에 담아 응답하며, 실패하면 에러 상태 코드와 메시지를 응답합니다.

참고 core/util/Resp.java. 공통 응답 형식

```
public record Resp<T>(Integer status, String msg, T body) {

    public static <B> ResponseEntity<Resp<B>> ok(B body) {
        return new ResponseEntity<>(new Resp<>(200, "성공", body), HttpStatus.OK);
    }

    public static ResponseEntity<?> fail(HttpStatus status, String msg) {
        return new ResponseEntity<>(new Resp<>(status.value(), msg, null), status);
    }
}
```

2.7 리포지토리(Repository)

먼저 데이터베이스에 접근하는 리포지토리를 만들어 보겠습니다. 리포지토리가 데이터베이스에 접근할 때 사용하는 도구가 **EntityManager**입니다. 이 객체는 스프링이 미리 빈(Bean)으로 등록해 두므로, 직접 만들 필요 없이 생성자로 주입받아 사용하면 됩니다.

board/BoardRepository.java 를 열어 아래와 같이 작성합니다.

실습 2 board/BoardRepository.java. 리포지토리 골격

```
@RequiredArgsConstructor
@Repository // 스프링이 빈으로 등록한다
public class BoardRepository {

    private final EntityManager em; // 의존성 주입

    // 아래 절에서 메서드를 하나씩 채운다
}
```

2.7.1 게시물 한 건 조회

EntityManager의 `find()` 메서드는 기본 키(PK)로 엔티티 한 건을 조회할 때 사용합니다. 첫 번째 인자에는 조회할 엔티티의 클래스 타입을, 두 번째 인자에는 찾으려는 기본 키 값을 넣어줍니다. 그러면 조건에 맞는 **Board** 엔티티 한 건을 반환합니다.

`board/BoardRepository.java`의 주석 자리에 아래 메서드를 작성합니다.

실습 3 board/BoardRepository.java. 기본 키로 한 건 조회

```
public Board findById(int boardId) {
    return em.find(Board.class, boardId);
}
```

`find()` 메서드는 데이터베이스에 다음과 같은 select 문을 전달합니다.

```
select id, content, created_at, title from board_tb where id = 1;
```

2.7.2 게시물 전체 조회

EntityManager에는 전체 목록을 한 번에 조회하는 전용 메서드가 없습니다. 따라서 전체 데이터를 가져오려면 **JPQL(Java Persistence Query Language)**을 사용해 직접 쿼리를 작성해서 실행해야 합니다.

실습 4 board/BoardRepository.java. JPQL로 전체 조회

```
public List<Board> findAll() {
```

```
return em.createQuery("select b from Board b", Board.class).getResultList();
}
```

`createQuery()` 에 JPQL 문자열과 반환 타입을 넘겨 쿼리를 생성한 뒤, `getResultList()` 를 호출해 실행합니다. 그러면 하이버네이트가 이 JPQL을 아래와 같은 실제 SQL로 번역해 줍니다.

```
select id, content, created_at, title from board_tb;
```

JPQL의 문법은 다음 절에서 따로 다루겠습니다.

2.7.3 게시물 저장

데이터를 조회할 때 `find()` 를 사용했다면, 새로운 데이터를 추가할 때는 `persist()` 메서드를 사용합니다. 새로 만든 엔티티 객체를 `persist()` 에 넘겨주기만 하면, 하이버네이트가 이를 분석해 INSERT 쿼리를 생성하고 데이터베이스에 저장합니다.

실습 5 board/BoardRepository.java. 새 게시물 저장

```
public void save(Board board) {
    em.persist(board);
}
```

`persist()` 가 데이터베이스에 전달하는 SQL은 다음과 같습니다. `created_at` 은 `@CreationTimestamp`가 채운 값이 그대로 들어갑니다.

```
insert into board_tb (content, created_at, title) values ('content3', now(), 'title 3');
```

2.7.4 게시물 수정

JPA에는 데이터를 수정하는 메서드가 없습니다. 조회해 온 엔티티 객체의 값만 변경하면 데이터베이스에 수정이 자동으로 반영됩니다. 이를 더티 체크(변경 감지)이라 부르며, 자세한 원리는 뒤에서 다루겠습니다.

2.7.5 게시물 삭제

게시글을 삭제할 때는 `remove()` 메서드를 사용합니다. 이 메서드에 지우고자 하는 엔티티를 넘겨주면 JPA가 이를 삭제 대상으로 표시하고, 데이터베이스에 삭제 쿼리를 전달합니다.

실습 6 board/BoardRepository.java. 게시물 삭제

```
public void delete(Board board) {  
    em.remove(board);  
}
```

`remove()` 가 데이터베이스에 전달하는 SQL은 다음과 같습니다.

```
delete from board_tb where id = 2;
```

2.8 JPQL

JPQL은 데이터베이스 테이블이 아닌 자바 엔티티와 필드 이름을 기준으로 작성하는 JPA 전용 SQL입니다. 작성된 JPQL은 실행 시점에 JPA가 SQL로 번역하여 데이터베이스에 전달합니다. SELECT, UPDATE, DELETE는 지원하지만 INSERT는 지원하지 않으므로, 새로운 데이터를 추가할 때는 앞서 살펴본 `persist()` 메서드를 사용합니다.

JPQL의 기본 형태는 테이블 이름 대신 엔티티 이름을 적고, 별칭을 사용해 대상을 가리키는 방식입니다. 앞서 전체 조회에 사용한 쿼리가 바로 이 형태입니다.

```
select b from Board b
```

일부 필드만 조회할 때는 별칭 뒤에 점(.)을 찍고 필드 이름을 적습니다. 이때 데이터베이스의 컬럼명 (`created_at`)이 아니라 엔티티의 필드명(`createdAt`)을 적어야 합니다.

```
select b.title, b.content from Board b
```

조건을 추가할 때는 `where` 절에 콜론(:)을 붙여 파라미터를 조건에 넣습니다.

```
select b from Board b where b.id = :id
```

수정과 삭제도 같은 방식으로 작성합니다.

```
update Board b set b.title = '제목 수정' where b.id = :id
```

```
delete from Board b where b.id = :id
```

2.9 영속성 컨텍스트

스프링의 **EntityManager**는 데이터베이스에 직접 SQL을 보내기 전에, 엔티티를 먼저 올려두고 관리하는 전용 공간을 운영하는데, 이 공간을 **영속성 컨텍스트(Persistence Context)** 라고 부릅니다.

`persist()` 나 `find()` 처럼 **EntityManager**를 통해 엔티티가 영속성 컨텍스트에 등록되면, **엔티티는 영속 상태가 됩니다**. 영속성 컨텍스트는 이 객체의 상태 변화를 지켜보고 있다가, 변경된 내용을 데이터베이스에 반영합니다.

영속성 컨텍스트의 특징은 크게 세 가지입니다.

2.9.1 캐싱

캐싱은 한 번 조회한 엔티티를 영속성 컨텍스트에 보관해 두었다가, **똑같은 엔티티를 다시 요청할 때 데이터베이스를 거치지 않고 바로 반환하는 기능입니다**.

처음 조회하는 게시글은 아직 영속성 컨텍스트에 없습니다. 따라서 데이터베이스에 SELECT 쿼리를 보내 데이터를 직접 조회해 온 뒤, 이를 영속성 컨텍스트에 보관하고 반환합니다.



반면 같은 게시글을 다시 조회할 때는 데이터베이스를 거치지 않습니다. 영속성 컨텍스트에 보관된 엔티티를 바로 반환하기 때문에, **한 트랜잭션 안에서** 여러 번 조회하더라도 SELECT 쿼리는 최초 한 번만 실행됩니다.

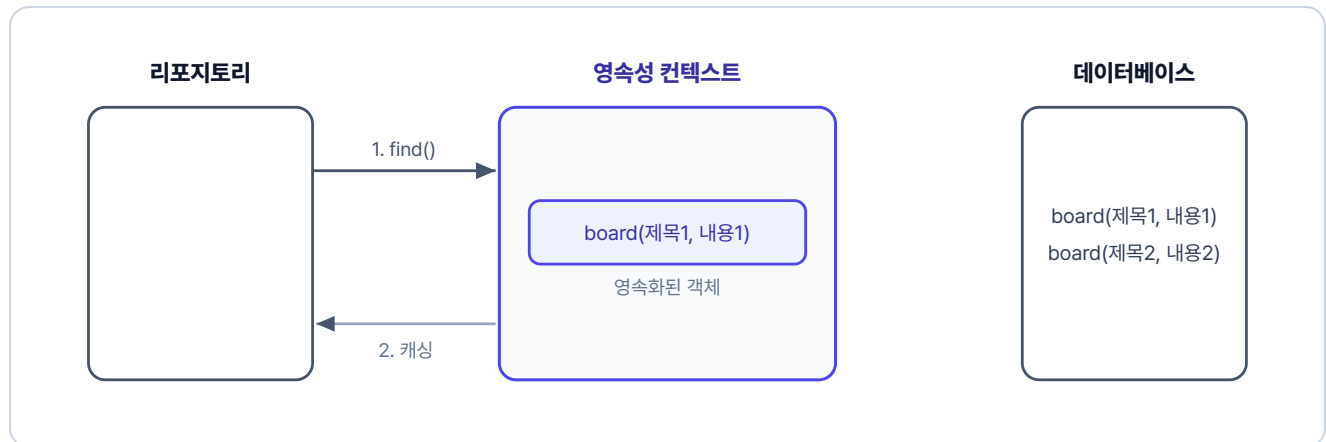


그림 2-21. 캐시 조회

같은 게시글을 두 번 조회해도 데이터베이스에 전달되는 SQL은 한 번뿐입니다.

```
select * from board_tb where id = 1;
```

트랜잭션이란?

트랜잭션(Transaction)은 여러 작업을 하나로 묶어 전부 반영하거나 전부 취소하는 작업 단위입니다.

스프링에서는 보통 서비스 계층의 메서드 하나가 한 트랜잭션 단위가 됩니다.

영속성 컨텍스트도 이 트랜잭션이 실행되는 동안에만 유지됩니다. 메서드가 정상적으로 끝나 트랜잭션이 종료되면 컨텍스트도 닫히고, 안에 있던 엔티티는 더 이상 관리되지 않습니다.

2.9.2 쓰기 지연

우리는 마트에서 물건을 고를 때마다 계산하지 않고, 장바구니에 담은 후 한 번에 결제합니다. 이와 같이 영속성 컨텍스트는 데이터를 변경하는 SQL을 곧바로 실행하지 않고 내부의 임시 공간(버퍼)에 모은 뒤 한 번에 데이터베이스로 전송합니다. 이런 방식을 쓰기 지연(Write Behind)이라고 합니다.

`persist()` 나 `remove()` 같은 메서드가 실행되면 JPA는 쿼리문을 버퍼에 저장한 후 트랜잭션이 성공하는 시점에 데이터베이스로 전달합니다.

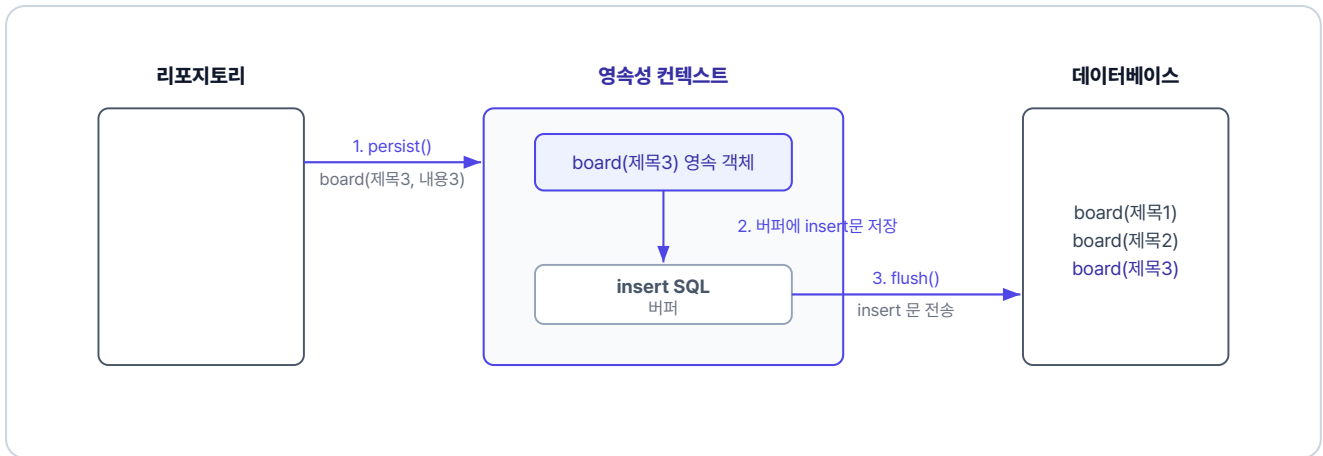


그림 2-22. 쓰기 지연

`persist()`로 저장한 게시글은 `flush()` 시점에 다음 SQL로 데이터베이스에 전달됩니다.

```
insert into board_tb (title, content, created_at) values ('제목3', '내용3', now());
```

flush()란?

flush()는 영속성 컨텍스트의 변경 사항을 데이터베이스에 동기화하기 위해, 버퍼에 쌓여 있던 SQL을 전송하는 메서드입니다. 개발자가 직접 호출하지 않아도 트랜잭션이 성공할 때 JPA가 실행합니다.

단, 기본 키 생성을 DB에 맡기는 IDENTITY 전략에서는 `persist()` 호출 즉시 INSERT 쿼리가 날아갑니다. DB가 만들어준 기본 키를 먼저 알아야 영속성 컨텍스트에 엔티티를 등록할 수 있기 때문입니다.

2.9.3 더티체킹

렌터카를 빌릴 때 직원이 차 상태를 사진으로 남겨 두었다가, 반납할 때 그 사진과 지금 차를 비교해서 달라진 곳을 찾아냅니다. 이와 같이 **영속성 컨텍스트는 엔티티가 영속 상태가 되는 순간의 값을 스냅샷으로 찍어 둡니다**. 이후 엔티티의 값이 변경되면 JPA가 스냅샷과 비교해 변경된 내용을 UPDATE 문으로 버퍼에 담아 둡니다. 이를 **더티체킹(Dirty Checking)**이라고 합니다.

수정할 게시글을 `find()`로 조회하면 영속화가 되며 이 상태를 스냅샷으로 남깁니다.



그림 2-23. 수정할 게시물 조회와 영속화

영속화된 엔티티의 값을 바꾸면 스냅샷과 달라집니다. 영속성 컨텍스트는 이 차이를 감지해 UPDATE 문을 버퍼에 만들어 두고, `flush()` 시점에 데이터베이스로 내보냅니다.

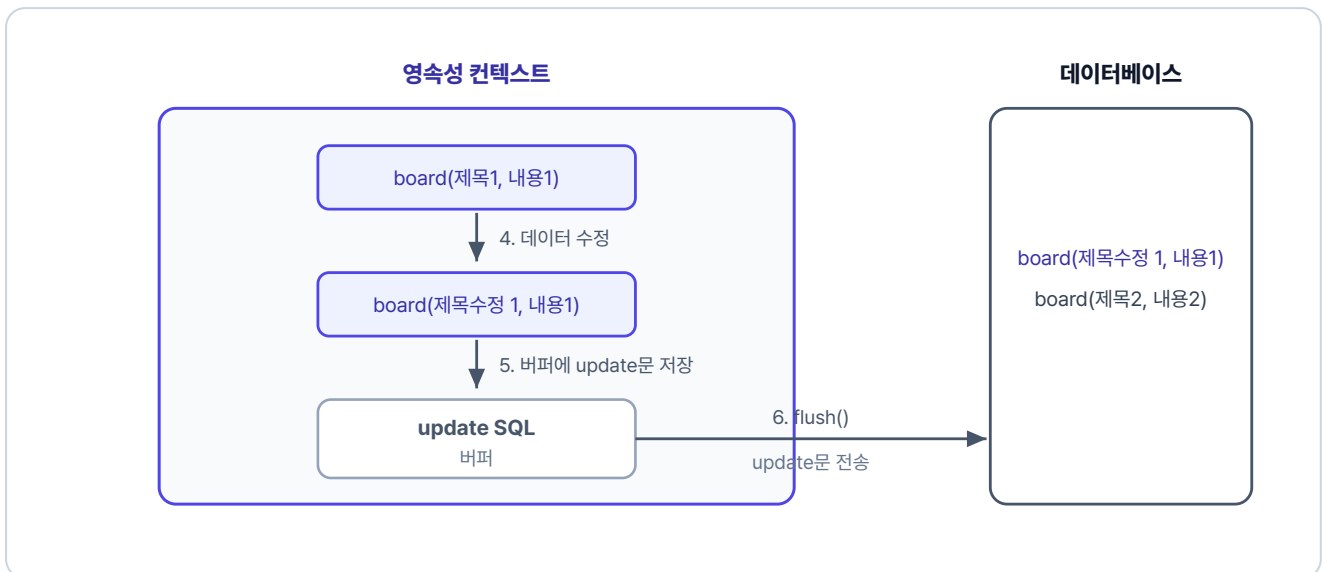


그림 2-24. 더티체킹

2.10 단위 테스트

리포지토리에 조회, 저장, 삭제 메서드 작성을 마쳤으니 기능이 의도대로 동작하는지 검증할 차례입니다. 이렇게 작성한 코드를 검증할 때는 **단위 테스트(Unit Test)** 를 사용합니다.

커피 머신을 예로 들어 보겠습니다. 커피 머신은 원두를 가는 분쇄기와 커피를 내리는 추출기로 구성됩니다. 두 기능이 하나로 결합되어 있다면, 커피가 정상적으로 나오지 않을 때 어느 쪽 문제인지 파악하기 어렵습니다.

반면 두 기능을 분리해 독립적으로 작동시키면 원인 파악이 쉬워집니다. 소프트웨어 역시 문제의 원인을 쉽게 찾기 위해 외부 의존성을 배제하고 가장 작은 기능 단위만 분리해 검증합니다. 이 방식이 단위 테스트입니다.

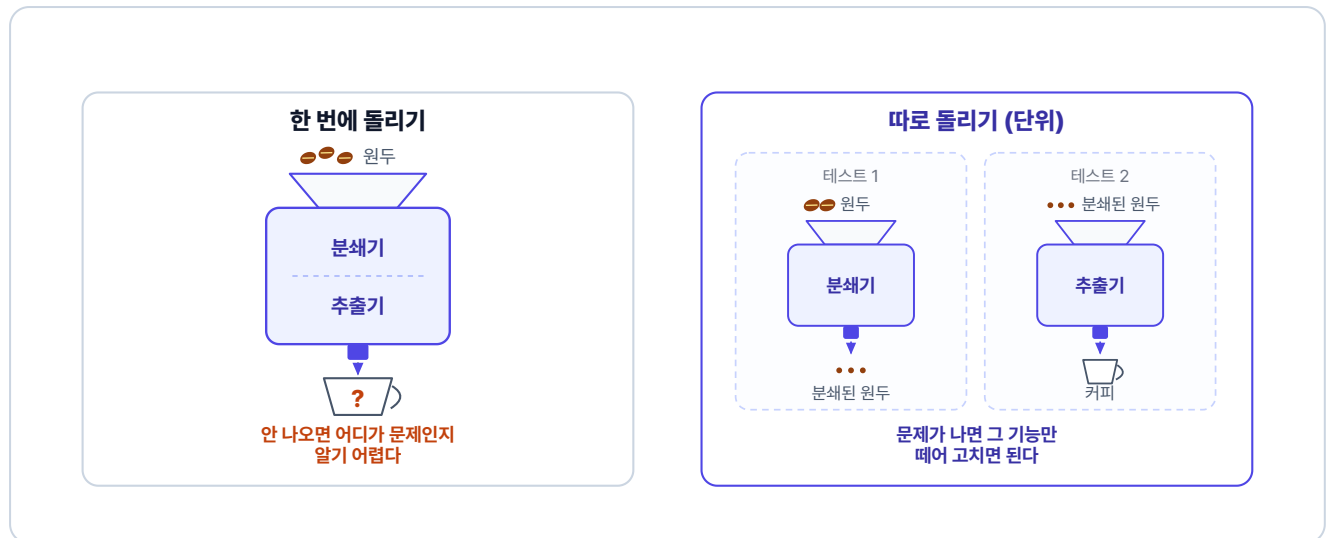


그림 2-25. 단위 테스트가 필요한 이유

자바에서는 이러한 단위 테스트 실행을 **JUnit**이 담당합니다. 검증할 메서드 위에 **@Test** 어노테이션만 추가하면 해당 메서드를 개별적으로 실행할 수 있습니다.

2.10.1 given-when-then 패턴

단위 테스트 코드는 일반적으로 **given-when-then**이라는 세 단계로 나누어 작성합니다. 테스트에 필요한 데이터를 준비하는 **given**, 검증할 대상 메서드를 호출하는 **when**, 그리고 실행 결과가 기대한 대로 나왔는지 확인하는 **then** 순서입니다.

본래 마지막 then 단계에서는 테스트 도구를 사용해 예상값과 실제 결과값을 코드로 비교하고 검증합니다. 하지만 이 책에서는 편의상 복잡한 검증 코드를 작성하는 대신, 실행 결과를 콘솔에 출력하고 눈으로 직접 확인하는 **eye** 단계로 대체하여 진행하겠습니다.



행위 주도 개발(Behavior Driven Development)

Given-When-Then 패턴은 사용자 관점에서 시스템이 **어떤 행위**를 하는지 시나리오 형태로 작성하는 행위 주도 개발 기법에서 유래했습니다. 테스트 코드가 마치 한 편의 설명서처럼 읽히기 때문에, 개발자뿐만 아니라 다른 협업자들도 요구사항과 테스트의 목적을 직관적으로 파악할 수 있습니다.

2.10.2 리포지토리 테스트 작성

이제 given-when-eye 형식에 맞춰 실제 테스트를 만들어 보겠습니다.

테스트 코드는 `src/test/java` 아래에 `src/main/java` 와 같은 패키지 구조로 둡니다. `board` 패키지에 `BoardRepositoryTest.java` 를 생성합니다.

`BoardRepositoryTest.java` 파일에는 아래와 같이 코드를 작성합니다.

실습 7 BoardRepositoryTest.java. 테스트 클래스 골격

```
@Import(BoardRepository.class) // 검증할 BoardRepository를 빈으로 등록한다
@DataJpaTest // 데이터베이스 연결과 EntityManager를 등록한다
public class BoardRepositoryTest {

    @Autowired // 등록된 빈을 주입받는다
    private BoardRepository boardRepository;

    @Autowired
    private EntityManager em;

    // 테스트 메서드를 하나씩 채운다
}
```

먼저 게시글 한 건 조회입니다. 테스트 메서드를 아래와 같이 작성합니다.

실습 8 BoardRepositoryTest.java. 한 건 조회

```
@Test
public void findById_test() {
    // given
    int id = 2;
    // when
    Board board = boardRepository.findById(id);
}
```

```
// eye
System.out.println("=====");
System.out.println("Board Title : " + board.getTitle());
System.out.println("Board Content : " + board.getContent());
}
```

```
Hibernate:
select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
from
    board_tb b1_0
where
    b1_0.id=?
=====
Board Title : title2
Board Content : content2
```

그림 2-27. 한 건 조회 실행 결과

전체 게시물 조회는 결과가 여러 개의 엔티티로 반환되므로, 이를 담기 위해 List 타입을 사용합니다.

실습 9 BoardRepositoryTest.java. 전체 조회

```
@Test
public void findAll_test() {
    // given

    // when
    List<Board> boards = boardRepository.findAll();
    // eye
    System.out.println("=====");
    System.out.println("Board Count : " + boards.size());
    System.out.println("Board 1 title : " + boards.get(0).getTitle());
    System.out.println("Board 2 content : " + boards.get(1).getContent());
}
```

```

Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
=====
Board Count : 2
Board 1 title :title1
Board 2 content :content2
=====

```

그림 2-28. 전체 조회 실행 결과

게시글 저장은 `save()` 메서드를 실행한 후 저장된 결과를 확인하기 위해 `findAll()` 을 한 번 더 사용합니다.

실습 10 BoardRepositoryTest.java. 저장

```

@Test
public void save_test() {
    // given
    Board board = new Board();
    board.setTitle("title3");
    board.setContent("content3");
    // when
    boardRepository.save(board);
    // eye
    List<Board> boards = boardRepository.findAll();
    System.out.println("=====");
    System.out.println("Board Count : " + boards.size());
    System.out.println("Board ID: " + boards.get(2).getId());
    System.out.println("Board Title: " + boards.get(2).getTitle());
    System.out.println("Board Content: " + boards.get(2).getContent());
}

```

```

Hibernate:
    insert
    into
        board_tb
        (content, created_at, title, id)
    values
        (?, ?, ?, default)
Hibernate:
    select
        b1_0.id,
        b1_0.content,
        b1_0.created_at,
        b1_0.title
    from
        board_tb b1_0
=====
Board Count : 3
Board ID: 3
Board Title: title3
Board Content: content3

```

그림 2-29. 저장 실행 결과

게시글 수정은 update 메서드를 만들지 않고 영속 상태 엔티티의 더티체킹을 사용합니다.

실제 서비스에서는 트랜잭션이 끝날 때 JPA가 `flush()` 를 실행하지만, 테스트에서는 검증하는 시점에 트랜잭션이 아직 끝나지 않았으므로 `flush()` 를 직접 호출합니다. 그리고 `clear()` 를 활용해 영속성 컨텍스트를 비운 후, 데이터베이스의 값을 조회합니다.

실습 11 BoardRepositoryTest.java. 수정과 더티체킹

```

@Test
public void update_test() {
    // given
    int id = 2;
    // when
    Board board = boardRepository.findById(id);
    board.setTitle("title-update");
    board.setContent("Update-test");
    em.flush();
    em.clear(); // 영속성 컨텍스트를 강제로 비운다
    // eye
    Board result = boardRepository.findById(id);
    System.out.println("=====");
    System.out.println("Board title : " + result.getTitle());
    System.out.println("Board content : " + result.getContent());
}

```

```
}
```

```
Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
  where
    b1_0.id=?
Hibernate:
  update
    board_tb
  set
    content=?,
    created_at=?,
    title=?
  where
    id=?
Hibernate:
  select
    b1_0.id,
    b1_0.content,
    b1_0.created_at,
    b1_0.title
  from
    board_tb b1_0
  where
    b1_0.id=?
=====
Board title : title-update
Board content : Update-test
```

그림 2-30. 수정 실행 결과

게시글 삭제 테스트를 아래와 같이 작성합니다.

실습 12 BoardRepositoryTest.java. 삭제

```
@Test
public void delete_test() {
    // given
    int id = 2;
    Board board = boardRepository.findById(id);
    // when
    boardRepository.delete(board);
    em.flush();
    // eye
    List<Board> boards = boardRepository.findAll();
    System.out.println("=====");
    System.out.println("Board count : " + boards.size());
}
```

```

Hibernate:
    delete
    from
        board_tb
    where
        id=?
Hibernate:
    select
        b1_0.id,
        b1_0.content,
        b1_0.created_at,
        b1_0.title
    from
        board_tb b1_0
=====
Board count : 1

```

그림 2-31. 삭제 실행 결과

이렇게 단위 테스트로 리포지토리만 따로 검증하면 서버를 실행하지 않아도 기능이 의도대로 동작하는지 확인할 수 있습니다.

2.11 게시물 목록

2.11.1 서비스

먼저 서비스를 작성합니다. 서비스는 리포지토리를 주입받고, `게시글목록()` 에서 리포지토리의 `findAll()` 을 호출해 결과를 반환합니다.

`board/BoardService.java` 를 열어 아래와 같이 작성합니다.

실습 13 board/BoardService.java. 게시물 목록

```

@RequiredArgsConstructor
@Service // 스프링이 빈으로 등록한다
public class BoardService {

    private final BoardRepository boardRepository; // 의존성 주입

    public List<Board> 게시물목록() {
        return boardRepository.findAll();
    }
}

```

```
}
```

2.11.2 컨트롤러

이어서 요청을 받을 컨트롤러를 작성합니다. 컨트롤러는 서비스를 주입받고, 서비스의 `게시글목록()` 을 호출해 결과를 반환합니다.

`board/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 14 board/BoardController.java. 게시글 목록

```
@RequiredArgsConstructor
@RestController
@RequestMapping("/api/boards") // 공통 주소
public class BoardController {

    private final BoardService boardService; // 의존성 주입

    @GetMapping
    public ResponseEntity<?> findAll() {
        List<Board> responseBoardList = boardService.게시글목록();
        return Resp.ok(responseBoardList);
    }
}
```

2.11.3 API 요청

애플리케이션을 실행합니다.

터미널 애플리케이션 실행

```
./gradlew bootRun
```

서버를 실행했으니, 이제 API를 호출해 잘 동작하는지 확인해 보겠습니다. 이 책에서는 브라우저에서 API를 호출하는 도구인 **Hoppscotch**(<https://hoppscotch.io/>)를 사용합니다.

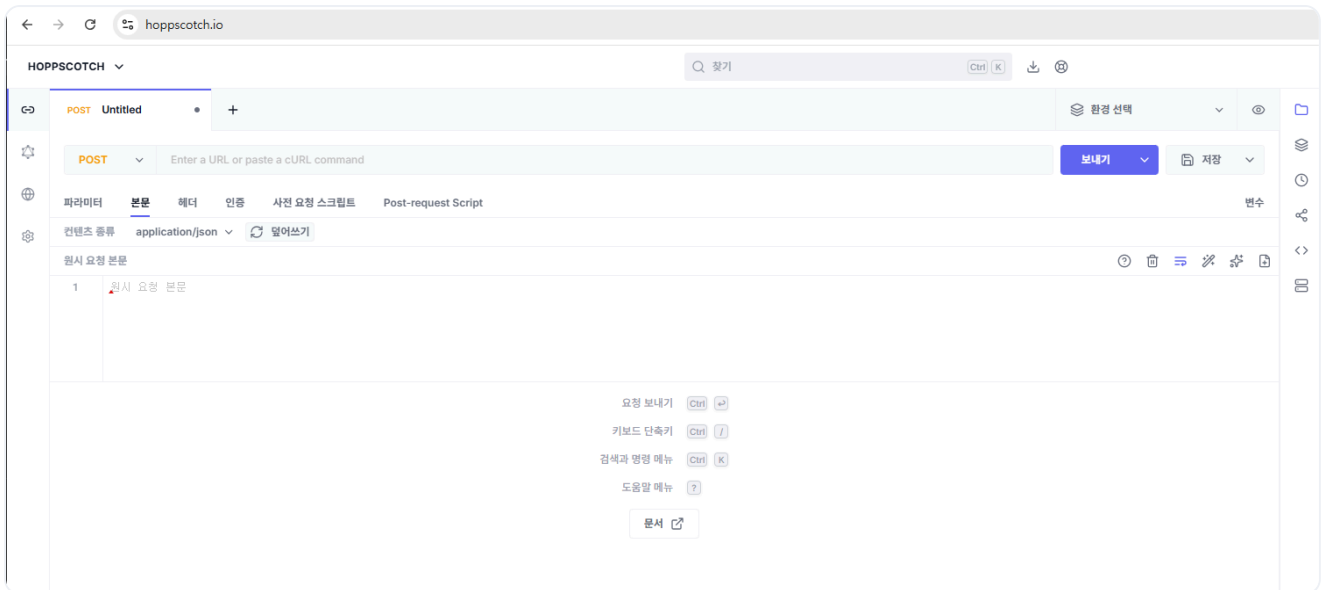


그림 2-32. Hoppscotch 화면

웹 브라우저는 보안 때문에 내 컴퓨터의 localhost로 바로 요청을 보내지 못합니다. 그래서 요청을 대신 전달해 주는 **Hoppscotch Browser Extension**을 Chrome 웹 스토어에서 설치하고, 설정 > Interceptor에서 익스텐션을 선택합니다.

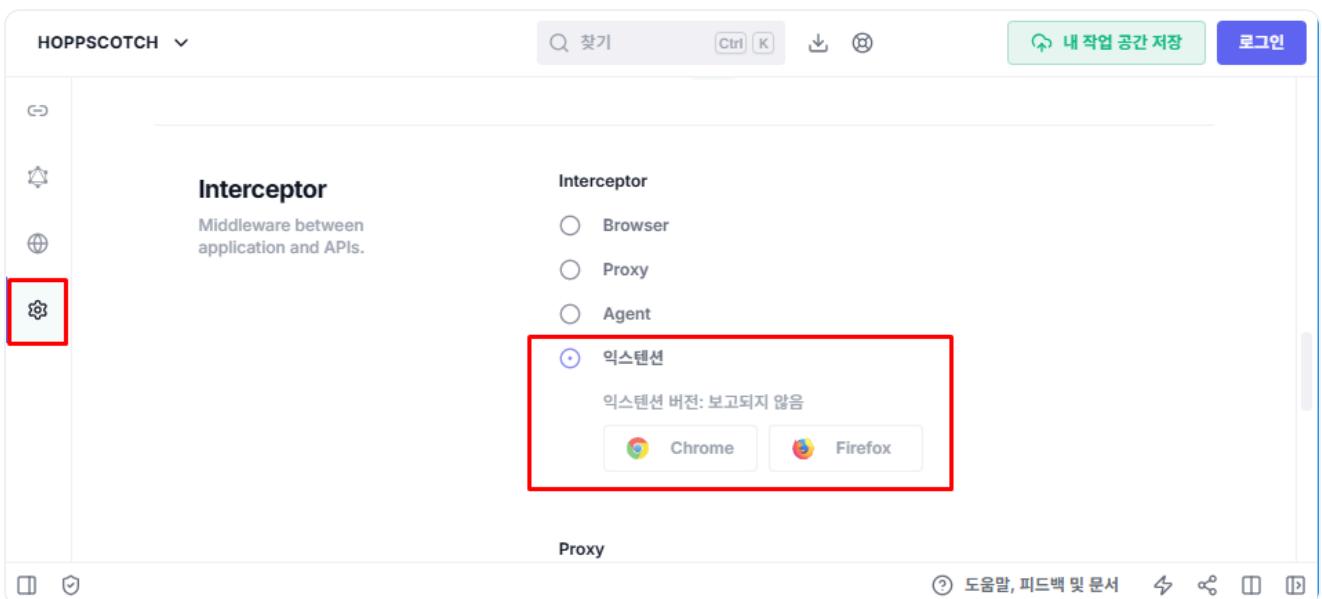


그림 2-33. Browser Extension 인터셉터 설정

확장 프로그램을 설치했다면 게시글 목록 API를 호출합니다.

Hoppscotch 게시글 목록 조회

GET `http://localhost:8080/api/boards`

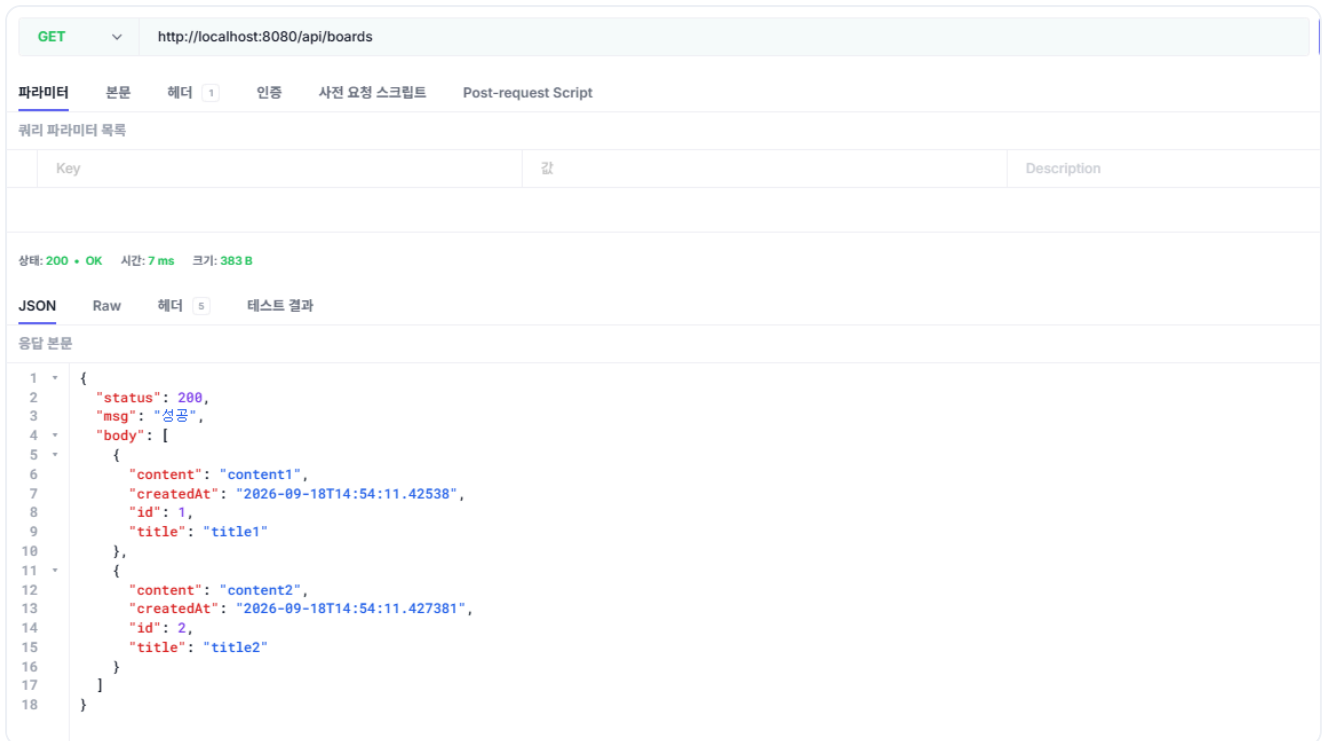


그림 2-34. 게시글 목록 응답

2.12 게시글 상세

2.12.1 서비스

게시글 상세 조회는 기본 키를 사용해 단일 데이터만 가져옵니다. 서비스에서 리포지토리의 `findById()` 를 호출하도록 `board/BoardService.java` 에 다음 메서드를 추가합니다.

실습 15 board/BoardService.java. 게시글 상세

```

public Board 게시글상세(Integer boardId) {
    return boardRepository.findById(boardId);
}

```

2.12.2 컨트롤러

컨트롤러는 URL 경로에 포함된 게시글 번호를 추출하여 서비스로 전달합니다. 경로 변수인 `{boardId}` 값은 `@PathVariable`을 사용해 가져옵니다.

`board/BoardController.java` 에 다음 메서드를 추가해 이를 구현합니다.

실습 16 board/BoardController.java. 게시물 상세

```
@GetMapping("/{boardId}")
public ResponseEntity<?> findById(@PathVariable("boardId") Integer boardId) {
    Board responseBoard = boardService.게시글상세(boardId);
    return Resp.ok(responseBoard);
}
```

서버를 실행한 뒤 1번 게시글의 상세 조회 API를 호출합니다.

Hoppscotch 게시물 상세 조회

GET <http://localhost:8080/api/boards/1>

The screenshot shows the Hoppscotch interface for a GET request to `http://localhost:8080/api/boards/1`. The response status is 200 OK, with a response time of 25 ms and a size of 293 B. The response body is a JSON object:

```
{
  "status": 200,
  "msg": "성공",
  "body": {
    "content": "content1",
    "createdAt": "2026-09-18T14:54:11.42538",
    "id": 1,
    "title": "title1"
  }
}
```

그림 2-35. 게시물 상세 응답

2.13 게시물 추가

2.13.1 서비스

게시글 추가는 데이터를 변경하는 작업입니다. 중간에 오류가 발생하더라도 데이터에 잘못 반영되지 않도록, 메서드 전체를 하나의 트랜잭션으로 묶어 처리합니다.

`board/BoardService.java`에 아래와 같이 메서드를 추가합니다.

실습 17 board/BoardService.java. 게시물 추가

```
@Transactional
public Board 게시물추가(Board requestBoard) {
    boardRepository.save(requestBoard);
    return requestBoard;
}
```

@Transactional은 메서드가 실행되는 동안 트랜잭션을 열어 두었다가, 정상으로 끝나면 변경을 반영하고 중간에 예외가 나면 되돌립니다.

2.13.2 컨트롤러

클라이언트의 요청이 들어오면 디스패처 서블릿이 담당 컨트롤러 메서드를 찾아 호출합니다. 파라미터에 **@RequestBody**가 있다면, 요청 바디에 담긴 JSON 데이터를 자바 객체로 변환해 매개변수로 전달합니다.

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 18 board/BoardController.java. 게시물 추가

```
@PostMapping
public ResponseEntity<?> save(@RequestBody Board requestBoard) {
    Board responseBoard = boardService.게시물추가(requestBoard);
    return Resp.ok(responseBoard);
}
```

저장한 게시물을 응답합니다. 클라이언트가 다시 조회하지 않아도 저장된 결과를 바로 확인할 수 있습니다.

제목과 내용을 요청 바디에 담아 게시물 추가 API를 호출합니다.

Hoppscotch 게시물 추가

```
POST http://localhost:8080/api/boards

{
  "title": "title3",
  "content": "content3"
}
```

```
}
```

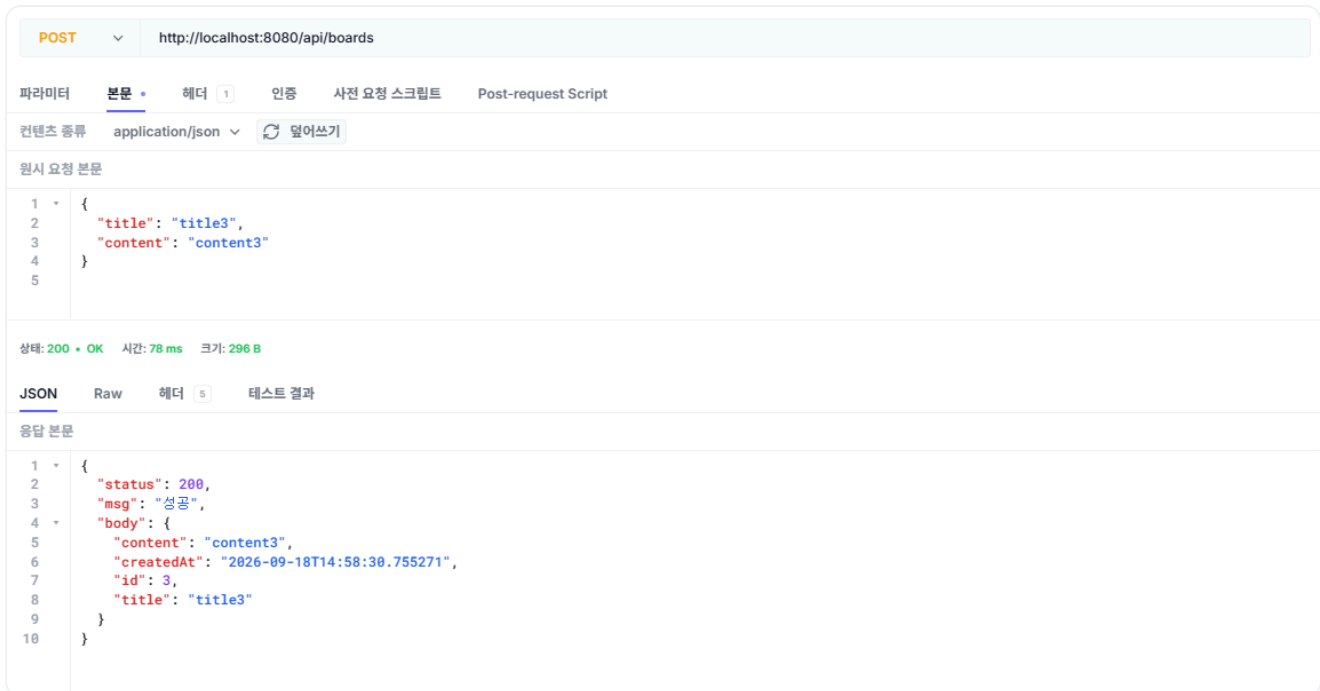


그림 2-36. 게시글 추가 응답

2.14 게시글 수정

2.14.1 서비스

`게시글수정()` 은 URL에 담긴 게시글 번호와 요청 바디로 들어온 값을 함께 전달받습니다.

`board/BoardService.java` 에 아래와 같이 메서드를 추가합니다.

실습 19 board/BoardService.java. 더티체킹으로 수정

```
@Transactional
public Board 게시글수정(Integer boardId, Board requestBoard) {
    // 1. 수정할 게시글을 조회해 영속 상태로 가져온다
    Board board = boardRepository.findById(boardId);
    // 2. 값만 바꾼다. save() 호출이 없다
    board.setTitle(requestBoard.getTitle());
    board.setContent(requestBoard.getContent());
    return board;
} // 트랜잭션이 끝나는 이 지점에서 변경이 반영된다
```

값을 변경하기만 했는데도, **@Transactional**이 끝날 때 `flush()`가 스냅샷과 현재 값을 비교해 달라진 엔티티를 UPDATE로 내보냅니다.

2.14.2 컨트롤러

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 20 board/BoardController.java. 게시물 수정

```
@PostMapping("/{boardId}")
public ResponseEntity<?> update(@PathVariable("boardId") Integer boardId,
                                @RequestBody Board requestBoard) {
    Board responseBoard = boardService.게시글수정(boardId, requestBoard);
    return Resp.ok(responseBoard);
}
```

바꿀 제목과 내용을 요청 바디에 담아 1번 게시글의 수정 API를 호출합니다.

Hoppscotch 게시물 수정

```
PUT http://localhost:8080/api/boards/1

{
  "title": "title-update",
  "content": "content-update"
}
```

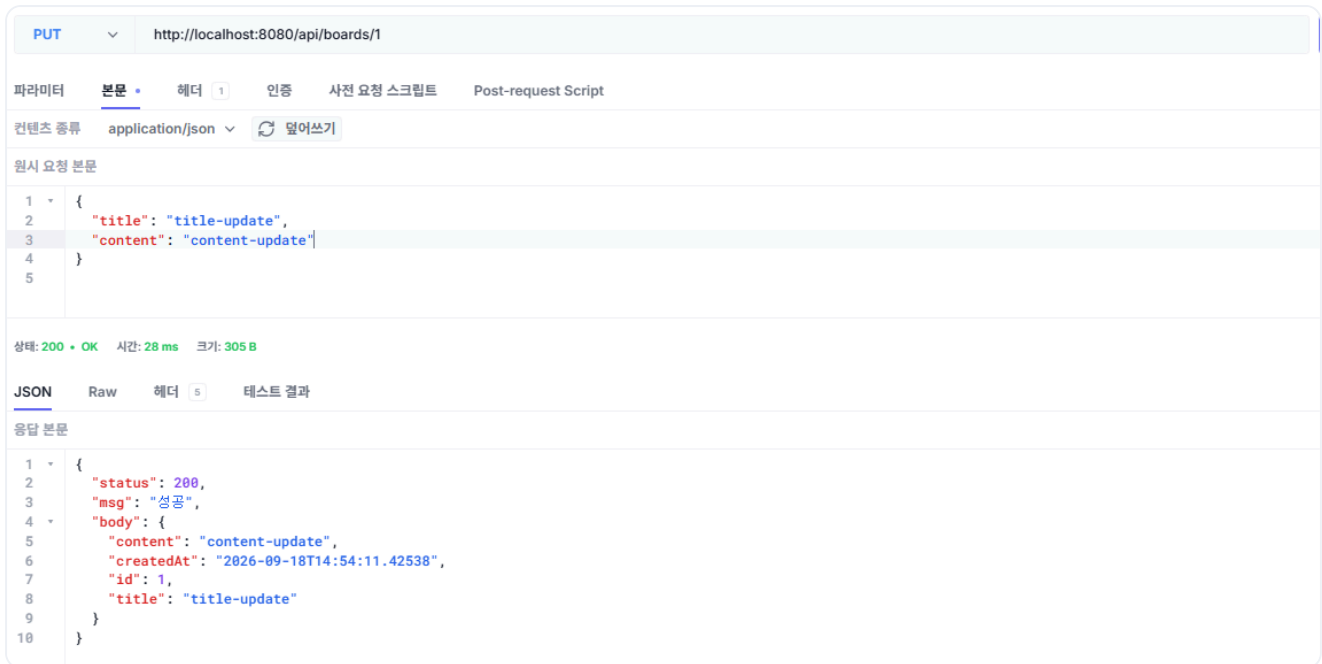


그림 2-37. 게시물 수정 응답

2.15 게시물 삭제

2.15.1 서비스

삭제는 지울 엔티티를 먼저 조회해 리포지토리에 전달합니다.

`board/BoardService.java`에 아래와 같이 메서드를 추가합니다.

실습 21 board/BoardService.java. 게시물 삭제

```
@Transactional
public void 게시물삭제(Integer boardId) {
    Board board = boardRepository.findById(boardId);
    boardRepository.delete(board);
}
```

2.15.2 컨트롤러

`board/BoardController.java`에 아래와 같이 메서드를 추가합니다.

실습 22 board/BoardController.java. 게시물 삭제

```
@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(@PathVariable("boardId") Integer boardId) {
    boardService.게시글삭제(boardId);
    return Resp.ok(null);
}
```

삭제는 반환할 데이터가 없으므로 `Resp.ok(null)` 로 성공만 응답합니다.

1번 게시글의 삭제 API를 호출합니다.

Hoppscotch 게시물 삭제

DELETE <http://localhost:8080/api/boards/1>

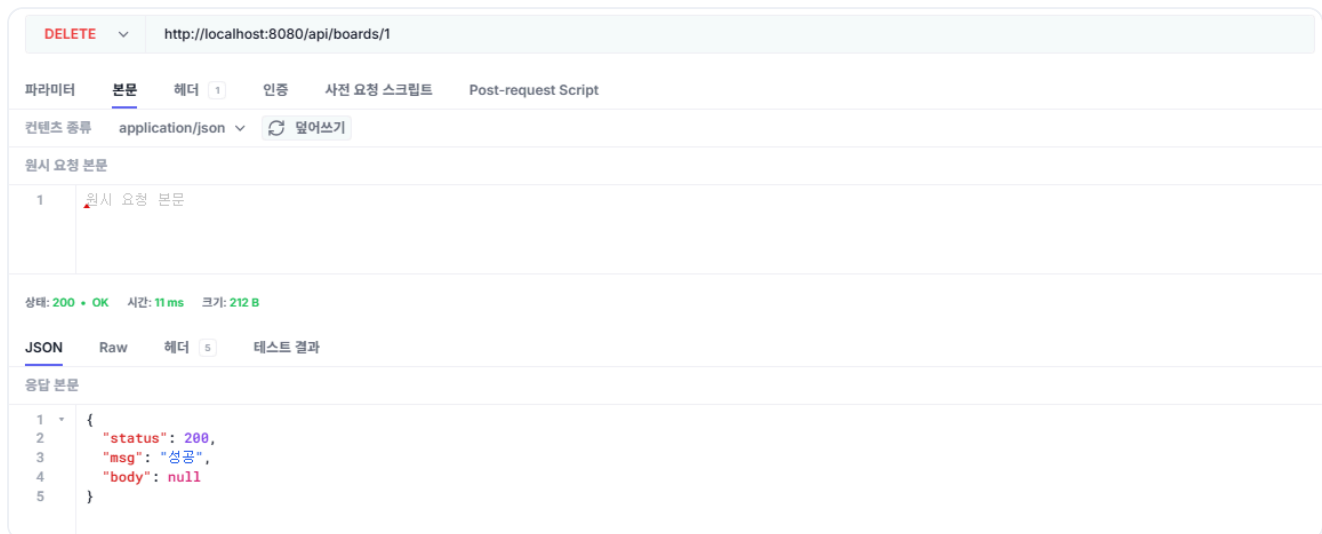


그림 2-38. 게시물 삭제 응답

스프링 프레임워크의 웹 계층 구조를 활용해 게시판의 기본 기능을 완성했습니다. 데이터베이스와 통신하는 리포지토리, 비즈니스 로직을 품은 서비스, 클라이언트의 요청을 받는 컨트롤러까지, 스프링의 구성 요소들이 어떻게 맞물려 하나의 애플리케이션으로 동작하는지 확인해 보았습니다.

다음 챕터에서는 클라이언트와 주고받을 데이터를 DTO로 분리하여 구조를 개선하고, 잘못된 조회 요청에 대비한 예외 처리를 다룹니다.

이것만은 기억하자

- **JPA는 객체와 테이블 사이를 변환합니다.** 테이블이 아니라 엔티티와 필드 이름으로 JPQL을 작성하면 JPA가 이를 SQL로 번역해 데이터베이스에 전달합니다.
- **영속성 컨텍스트는 조회한 엔티티를 트랜잭션 동안 관리합니다.** 같은 엔티티를 다시 조회하면 캐시에 있는 것을 반환하고, 쿼리는 버퍼에 모았다가 `flush()` 시점에 내보냅니다.
- **애플리케이션은 컨트롤러, 서비스, 리포지토리 세 계층으로 나눕니다.** 컨트롤러는 요청을 받고, 서비스는 트랜잭션 내에서 비즈니스 로직을 처리하며, 리포지토리는 데이터베이스를 다룹니다.